Tarefa Gauss-Seidel - Cálculo Numérico

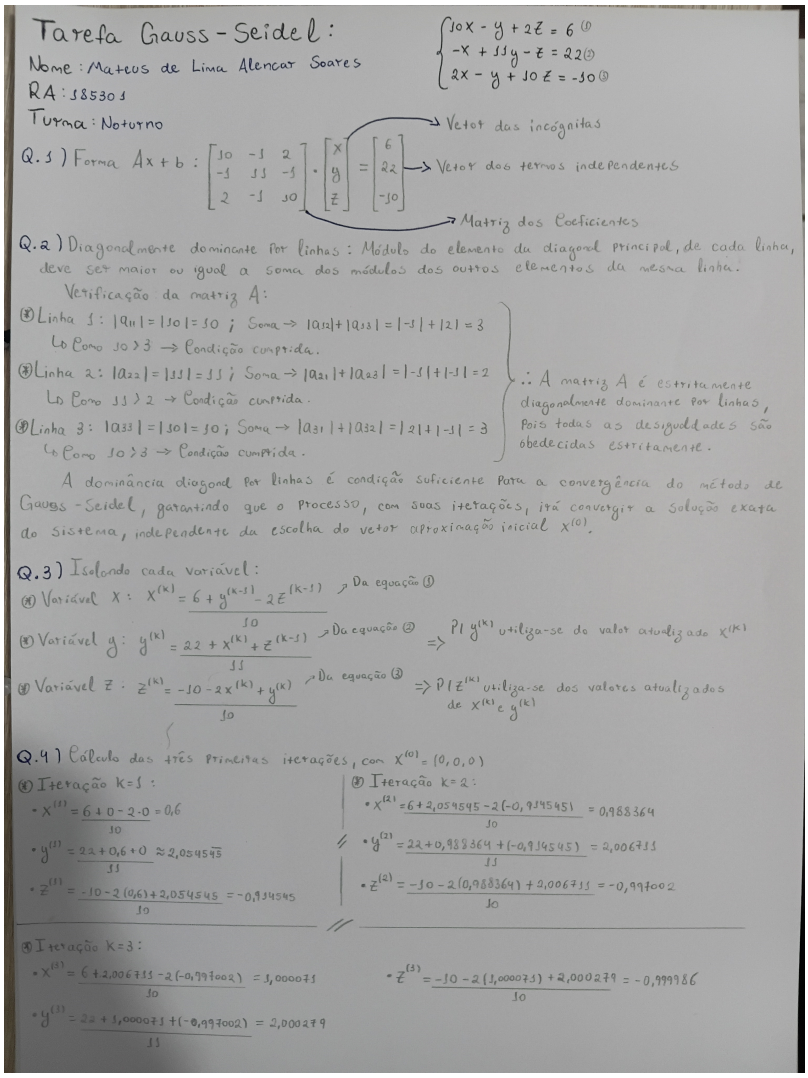
Nome: Mateus de Lima Alencar Soares
RA: 185301
Turma: Noturno

Questões 1 a 4 - Resolução manual:

```
from IPython.display import Image, display

# Substitua pelo nome exato do ficheiro que carregou
caminho_imagem = 'Questoes.jpg'

# Exibe a imagem com largura ajustada (600px)
display(Image(filename=caminho_imagem, width=600))
```



Questão 5 - Implementação em Python do Método de Gauss-Seidel:

```
import numpy as np

# ---------------------------------------------------------------------
# 1. Definição do Sistema Linear Ax = b
# ---------------------------------------------------------------------
A = np.array([
    [10.0, -1.0,  2.0],
    [-1.0, 11.0, -1.0],
    [ 2.0, -1.0, 10.0]
])

b = np.array([6.0, 22.0, -10.0])
```

```
# Configurações iniciais
x = np.array([0.0, 0.0, 0.0])  # Aproximação inicial x^(0)
tol = 1e-5                      # Tolerância de erro (10^-5)
max_iter = 100                  # Limite de segurança de iterações

# Listas para armazenar o histórico (usadas nas Questões 6 e 7)
historico_x = [x.copy()]
historico_erros = []

# ---------------------------------------------------------------------
# 2. Execução do Método de Gauss-Seidel
# ---------------------------------------------------------------------
print("--- Iniciando o Método de Gauss-Seidel ---")

for k in range(1, max_iter + 1):
    x_anterior = x.copy()

    # Atualizações com isolamento das incógnitas (usa valores mais recentes)
    x[0] = (6.0 + x[1] - 2.0 * x[2]) / 10.0
    x[1] = (22.0 + x[0] + x[2]) / 11.0
    x[2] = (-10.0 - 2.0 * x[0] + x[1]) / 10.0

    # Armazena o vetor aproximado da iteração k
    historico_x.append(x.copy())

    # Cálculo do erro: norma do máximo da diferença entre iterações
    erro = np.max(np.abs(x - x_anterior))
    historico_erros.append(erro)

    # Verificação do critério de parada
    if erro < tol:
        iteracoes_realizadas = k
        print(f"Convergência alcançada na iteração k = {k}!")
        break

print(f"\nSolução aproximada obtida:")
print(f"x = {x[0]:.6f}")
print(f"y = {x[1]:.6f}")
print(f"z = {x[2]:.6f}")
```

```
--- Iniciando o Método de Gauss-Seidel ---
Convergência alcançada na iteração k = 6!

Solução aproximada obtida:
x = 1.000000
y = 2.000000
z = -1.000000
```

Questão 6 - Tabela Organizada dos Resultados:

```
import pandas as pd
import numpy as np

# Converte o histórico de pontos para um array NumPy
historico_matriz = np.array(historico_x)

# Monta a estrutura da tabela utilizando o símbolo de infinito '∞'
dados_tabela = {
    'k': list(range(len(historico_matriz))),
    'x^(k)': historico_matriz[:, 0],
    'y^(k)': historico_matriz[:, 1],
    'z^(k)': historico_matriz[:, 2],
    '||x^(k) - x^(k-1)||∞': ['-'] + [f"{e:.6e}" for e in historico_erros]
}

df_resultados = pd.DataFrame(dados_tabela)

# Exibe a tabela formatada no Google Colab
print("--- TABELA DE ITERAÇÕES ---")
display(df_resultados)
```

```
--- TABELA DE ITERAÇÕES ---
```

	k	x^(k)	y^(k)	z^(k)	||x^(k) - x^(k-1)||∞
0	0	0.000000	0.000000	0.000000	-
1	1	0.600000	2.054545	-0.914545	2.054545e+00
2	2	0.988364	2.006711	-0.997002	3.883636e-01
3	3	1.000071	2.000279	-0.999986	1.170777e-02
4	4	1.000025	2.000004	-1.000005	2.755404e-04
5	5	1.000001	2.000000	-1.000000	2.389216e-05
6	6	1.000000	2.000000	-1.000000	1.262607e-06

Próximas etapas: Gerar código com `df_resultados` New interactive sheet

Questão 7 - Gráficos dos Resultados Obtidos:

a) Evolução de $x^{(k)}$, $y^{(k)}$ e $z^{(k)}$ em função de k

```
import matplotlib.pyplot as plt
import numpy as np

# Converter histórico de iterações para matriz
historico_matriz = np.array(historico_x)
k_eixo = np.arange(len(historico_matriz))

# Solução analítica/exata para referência
solucao_exata = [1.0, 2.0, -1.0]
cores = ['#1f77b4', '#ff7f0e', '#2ca02c']  # Azul, Laranja, Verde

fig, ax = plt.subplots(figsize=(11, 6), dpi=120)

# Plot das curvas de cada variável
marcadores = ['o', 's', '^']
rotulos = ['$x^{(k)}$', '$y^{(k)}$', '$z^{(k)}$']

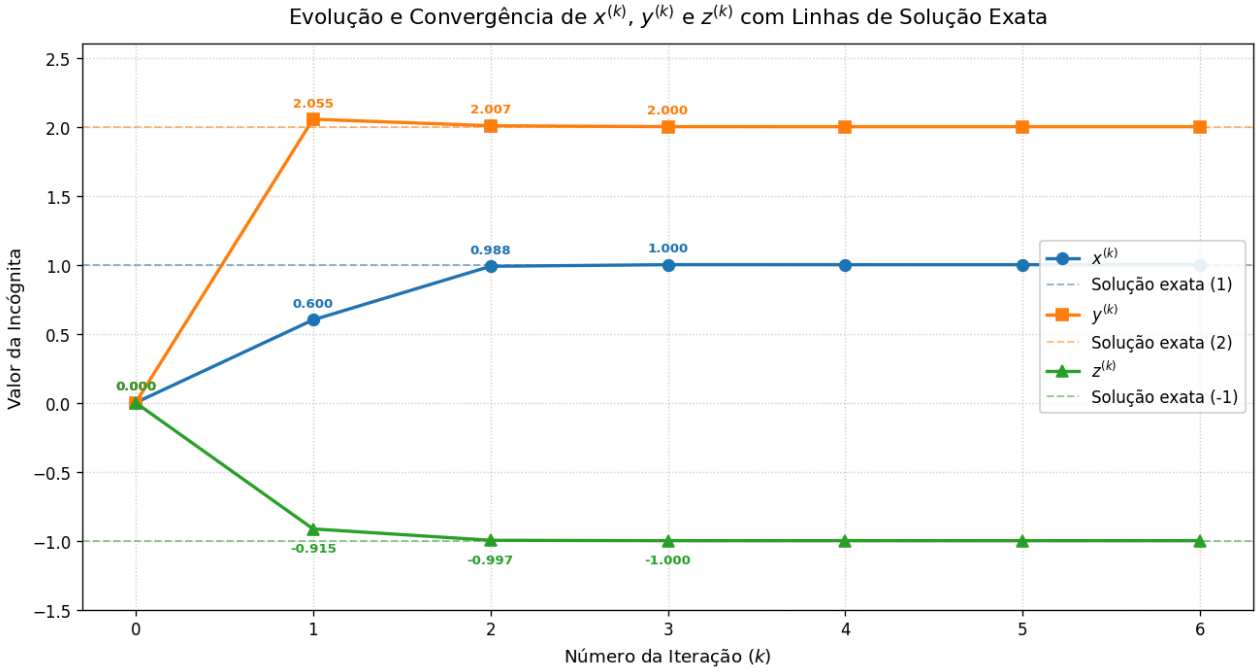
for i in range(3):
    # Curva da variável
    ax.plot(k_eixo, historico_matriz[:, i], marker=marcadores[i], color=cores[i],
            linewidth=2, markersize=7, label=rotulos[i], zorder=3)

    # Linha tracejada da solução exata
    ax.axhline(y=solucao_exata[i], color=cores[i], linestyle='--', alpha=0.5,
               linewidth=1.2, label=f'Solução exata ({solucao_exata[i]:.0f})' if k_eixo[0]==0 else "")

    # Adicionar anotações numéricas dos pontos até k = 3 para não poluir
    for k_val in range(min(4, len(k_eixo))):
        val = historico_matriz[k_val, i]
        ax.annotate(f'{val:.3f}',
                    (k_val, val),
                    textcoords="offset points",
                    xytext=(0, 8 if val >= 0 else -14),
                    ha='center', fontsize=8, fontweight='bold', color=cores[i])

# Ajustes do gráfico
ax.set_title('Evolução e Convergência de $x^{(k)}$, $y^{(k)}$ e $z^{(k)}$ com Linhas de Solução Exata', fontsize=13, pad=1
ax.set_xlabel('Número da Iteração ($k$)', fontsize=11)
ax.set_ylabel('Valor da Incógnita', fontsize=11)
ax.set_xticks(k_eixo)
ax.set_ylim(-1.5, 2.6)
ax.grid(True, linestyle=':', alpha=0.6)
ax.legend(loc='center right', fontsize=10, framealpha=0.9)

plt.tight_layout()
plt.show()
```



b) Erro entre iterações em escala logarítmica

```
plt.figure(figsize=(9, 4.5))

k_erros = np.arange(1, len(historico_erros) + 1)

# Plot do erro em escala semi-logarítmica (eixo Y logarítmico)
plt.semilogy(k_erros, historico_erros, 'ro-', linewidth=2, label=r'$||\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}||_{\infty}$')

# Linha de referência da tolerância
plt.axhline(y=tol, color='green', linestyle='--', linewidth=1.5, label=f'Tolerância ({tol})')

plt.title('Erro entre Iterações em Escala Logarítmica', fontsize=12)
plt.xlabel('Número da Iteração ($k$)', fontsize=11)
plt.ylabel('Norma do Erro (Escala Logarítmica)', fontsize=11)
plt.xticks(k_erros)
plt.grid(True, which="both", linestyle='--', alpha=0.6)
plt.legend(fontsize=11)
plt.tight_layout()
plt.show()
```

